Graph-Based

By Yiyang Liu

Process Review

①Pre-processing

②Tokenization

③Dimensionality Reduction

④Neighborhood Formation

⑤Graph Generation

⑥Community Detection

⑦Ranking Products within Graph Communities

⑧Generating Recommendations

IF WE HAVE A REAL DATASET, DO WE NEED SOME NEW METHODS?

A REAL DATA SET

	Food_ID	Describe
2	1	white balsamic vinegar, lemon juice, lemon rind, red chillies, garlic cloves (crushed), olive oil, summer squash (zucchini), sea salt, black pepper, basi
3	2	olive oil, chicken mince, garlic (minced), onion, salt, black pepper, carrot, cabbage, green onions, sweet chilli sauce, peanut butter, ginger, soy sauce
4	3	almonds whole, egg white, curry leaves, salt, sugar (fine grain), red chilli powder
5	4	vinegar, honey/sugar, soy sauce, salt, garlic cloves (minced), chilli pepper (sliced), green papaya, carrot (peeled), cucumbers, mint leaves, toasted pe
6	5	christmas dry fruits (pre-soaked), orange zest, lemon zest, jaggery syrup, almond flour, apple, butter (softened), eggs
7	6	japanese curry, sticky rice, cheese inside rice, barley salsa, wasabi mayo, red capsicum cube (cut), yellow capsicum cube (cut), green capsicum cube (
8	7	almonds, eggs, granulated sugar, bittersweet chocolate, unsalted butter, flour, baking powder, castor sugar, icing sugar
9	8	lamb bones (preferably shank and shoulder), onions, celery, ginger, garlic, carrot, chargrilled red/yellow/green bell peppers (quartered), whole spic
10	9	vegetable stock, skimmed milk, toasted almonds (powdered), butter, flour, salt and pepper, nutmeg, almond essence, toasted almond flakes
11	10	vegetable stock, broccoli, ground almonds (toasted), skimmed milk, salt, freshly ground black pepper
12	11	uncooked quinoa, water, red onion, cucumber (diced), purple cabbage, avocado (ripened and diced), orange, shelled edamame (defrosted), unsweet
13	12	young corn on the cob, honey, lemon juice, garlic cloves (smashed), celery stalk, chives, carrot, salt, paprika powder, parsley, plastic bag, food therm
14	13	fresh strawberries, honey, low fat yogurt, watermelon, chia seeds
15	14	fresh raspberries, ripe banana, almond, fresh peach slices, low fat yogurt, fresh raspberry, peach fruit slices, almonds
16	15	almond flakes, onion, poha, cranberries (frozen/ dried), salt, oil, curry leaves, green chilies, fresh coconut
17	16	almond slivers, raw banana (boiled), almond paste, cooking cream, refined oil, mace powder, cardamom powder , ginger garlic paste, garam masal
18	17	almonds (crushed), tomato, garlic cloves, basil sprig, lemon, salt, pepper, for namak para:, refined flour, sugar, salt, olive oil, water
19	18	khoya, sweetener (optional), almonds (crushed)
20	19	whole wheat flour (atta), refined flour (maida), garlic cloves (crushed), salt, red chilli powder, chaat masala, cumin powder, tomato puree, ghee
21	20	oats, ragi flour (bhakri atta), wheat flour, rice flour, urad dal flour (dry roast and grind the dal to a fine powder), cumin seeds, green chillies, ginger,
22	21	apples, milk, sugar, green cardamoms, almonds (blanched), pistachios (blanched)

- 400 different comments
- different food names
- some stop words and not_food words

HOW CAN WE grab the keywords?

dictionary to match?

Grab all key words

white balsamic vinegar, lemon juice, lemon rind, red chillies, garlic cloves (crushed), olive oil, summe
olive oil, chicken mince, garlic (minced), onion, salt, black pepper, carrot, cabbage, green onions, swe
almonds whole, egg white, curry leaves, salt, sugar (fine grain), red chilli powder
vinegar, honey/sugar, soy sauce, salt, garlic cloves (minced), chilli pepper (sliced), green papaya, car
christmas dry fruits (pre-soaked), orange zest, lemon zest, jaggery syrup, almond flour, apple, butter (
japanese curry, sticky rice, cheese inside rice, barley salsa, wasabi mayo, red capsicum cube (cut), yell
almonds, eggs, granulated sugar, bittersweet chocolate, unsalted butter, flour, baking powder, castor

DELETE

- all stop words
- all not food words
- all adjective
- .....

ONLY KEEP FOOD WORDS

Creat a food dictionary

```
food_items = [
    "Apple", "Banana", "Cherry", "Date", "Elderberry", "Fig", "Grape", "Honeydew melon", "Kiwi", "Lemon",
"Mango", "Nectarine", "Orange", "Papaya", "Quince", "Raspberry", "Strawberry", "Tomato", "Ugli fruit",
"Vanilla", "Watermelon", "Xigua", "Yellowtail", "Zucchini", "Almonds", "Bacon", "Cod", "Duck", "Eel",
"Flounder", "Grouper", "Halibut", "Ice cream", "Jambalaya", "Kale", "Lamb", "Mussels", "Noodles",
"Oysters", "Pork", "Quail", "Ribs", "Salmon", "Tuna", "Urchin", "Veal", "Waffles", "Xacuti", "Yogurt",
"Ziti", "Artichoke", "Broccoli", "Carrot", "Donut", "Eggplant", "French fries", "Green beans",
"Hamburger", "Icing", "Jelly", "Ketchup", "Lasagna", "Meatballs", "Nachos", "Olive oil", "Pizza",
"Quiche", "Roast beef", "Sushi", "Turkey", "Upside-down cake", "Vinegar", "Wonton", "Xiaolongbao",
"Yam", "Za'atar", "Avocado", "Brisket", "Cheese", "Dumpling", "Escargot", "Falafel", "Gnocchi",
"Hot dog", "Indian curry", "Jalapeno", "Kimchi", "Lobster", "Macaroni", "Nutella", "Octopus",
"Paella", "Quesadilla", "Ravioli", "Steak", "Truffle", "Udon", "Vermicelli", "Whelk", "Xo sauce",
"Yucca", "Zereshk", "Bagel", "Crepes", "Durian", "Eggs Benedict", "Frittata", "Gelato", "Hummus",
"Irish stew", "Jerk chicken", "Kofta", "Linguine", "Moussaka", "Nori", "Onion rings", "Pancakes",
"Qorma", "Ratatouille", "Souffle", "Tacos", "Umeboshi", "Vada", "Walnut", "Xouba", "Yellow pepper",
"Zafrani pulao", "Acorn squash", "Biryani", "Corn", "Dosa", "Enchilada", "Fudge", "Gazpacho",
"Hash browns", "Italian bread", "Jambon", "Knish", "Latte", "Muffin", "Neapolitan pizza", "Oregano",
"Pesto", "Quinoa", "Rolls", "Sashimi", "Tempura", "Unagi", "Vindaloo", "Wasabi", "Xingren tofu",
"Yellowfin tuna", "Zuccotto", "Acai bowl", "Baklava", "Cinnamon roll", "Deviled eggs", "Empanada",
"Fish tacos", "Guacamole", "Hot chocolate", "Iceberg lettuce", "Jackfruit", "Kumquat", "Liver",
"Mozzarella", "Nougat", "Oatmeal", "Pumpkin pie", "Quavers", "Rhubarb pie", "Spaghetti", "Tiramisu",
"Ube", "Vanilla bean", "White chocolate", "Xinomavro grapes", "Yellow cake", "Zwieback", "Arugula",
"Beef stew", "Chicken", "Daiquiri", "Edamame", "French toast", "Grits", "Horseradish", "Israeli couscous",
"Jelly beans", "Kielbasa", "Lima beans", "Marzipan", "Nectar", "Oxtail", "Prosciutto", "Queso",
"Roasted nuts", "Sorbet", "Tagliatelle", "Umbrella fruit", "Vodka sauce", "Wild rice", "Ximenia fruit",
"Yellow squash", "Zampone", "Anchovies", "Beignets", "Churros", "Dragon fruit", "Escalope", "Fajitas",
"Gumbo", "Hash", "Italian sausage", "Jollof rice", "Kangaroo meat", "Lentils", "Marmalade", "Nuggets",
"Omelette", "Pulled pork", "Quail eggs", "Risotto", "Sausage", "Tofu", "Ugli", "Venison", "Wheat bread",
"Xavier steak", "Yeast", "Zabaione", "Apricot", "Blackberry", "Clementine", "Damson", "Egg", "Fig jam",
"Gooseberry", "Haricot beans", "Idli", "Jicama", "Kipper", "Lychee", "Mulberry", "Nettle", "Okra",
"Pear", "Quark", "Radish", "Seaweed", "Tangerine", "Umbu", "Victoria plum", "Wild boar", "Xouba fish",
"Yucca root", "Ziti pasta", "Arrowroot", "Bok choy", "Cranberry", "Dandelion greens", "Endive", "Fava beans",
"Goat cheese", "Hake", "Ice pop", "Juniper berries", "Kale chips", "Linguica", "Mango chutney", "Nashi pear",
"Oxtail soup", "Pineapple", "Quail breast", "Rabbit", "Scallops", "Tabbouleh", "Urad dal", "Vongole",
"Watercress", "Xigua melon", "Yogurt cheese", "Zuppa toscana", "Asparagus", "Blue cheese", "Caviar",
"Duck confit", "Elephant ear", "Filet mignon", "Goose", "Halva", "Indian paneer", "Jerusalem artichoke",
"Kumera", "Lobster bisque", "Mascarpone", "Nasi goreng", "Oyster sauce", "Peking duck", "Quark cheese",
"Roquefort", "Sablefish", "Tahini", "Uthappam", "Venetian liver", "Welsh rarebit", "Xanthan gum",
"Yam bean", "Zuppa inglese", "Anise", "Baba ghanoush", "Cassava", "Dorade", "Escarole", "Foie gras",
"Guava", "Haddock", "Israeli salad", "Java apple", "Kale stew", "Lamb shank", "Macadamia nuts", "Nashi",
"Oregano oil", "Parsnip", "Quail pie", "Ramp", "Sauerkraut", "Tapioca", "Urchin roe", "Vanilla extract",
"Walnut oil", "Xanthia cocktail", "Yellow beet", "Zucchini bread", "Aubergine", "Black truffle", "Cactus",
"Date palm", "Eel sushi", "Flaxseed", "Gingerbread", "Horse mackerel", "Italian truffle", "Jack cheese",
```

USE the food item list to match with our new dataset which ***only*** has food names

Complexed questions

```
00]: from sklearn.feature_extraction.text import TfidfVectorizer

# Create a TfidfVectorizer object
vectorizer = TfidfVectorizer()

# Fit and transform the 'Processed_Keywords' data into a TF-IDF matrix
tfidf_matrix = vectorizer.fit_transform(data['Processed_Keywords'])

# Convert the TF-IDF matrix to a pandas DataFrame for better readability
feature_names = vectorizer.get_feature_names_out()
df_tfidf = pd.DataFrame(tfidf_matrix.toarray(), columns=feature_names)

# Display the DataFrame with TF-IDF scores
df_tfidf.head()
```

00]:

	aall	acorn	adobo	agent	aioli	ajinomoto	ajwain	alfredo	almond	anardana	...	yeast	yello	yogurt	yok	yolk	youg
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	...	0.0	0.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	...	0.0	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.752191	0.0	...	0.0	0.0	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	...	0.0	0.0	0.0	0.0	0.0	
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	...	0.0	0.0	0.0	0.0	0.0	

5 rows × 612 columns

Questions needed to be solved:

- how can we get a full food dictionary
- how can we get a perfect matrix?(because every column has differnet numbers of food names)
- so how can we match this colums to make a perfect matrix?

Let try another case to corret some mistakes we made at last presentations.

Some mistakes

Tokenization

Main method: Convert Input data to Vector

"responsive"	"efficient"	"reasonable price"	"good service"
"good service"	"attitude"	"very good",	"high satisfaction"
"responsive"	"efficient"	"good service"	"general"

```python
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd


feedbacks = [
    "responsive", "efficient", "reasonable price", "good service",
    "good service", "attitude", "very good", "high satisfaction"
    "responsive", "efficient", "good service", "general"]

# creat TF-IDFvector
vectorizer = TfidfVectorizer()

# convert feedback data into a vectorized form
tfidf_matrix = vectorizer.fit_transform(feedbacks)

# Get keywords
feature_names = vectorizer.get_feature_names_out()

# convert TF-IDF matrix toPandas DataFrame
df_tfidf = pd.DataFrame(tfidf_matrix.toarray(), columns=feature_names)

# DataFrame
print(df_tfidf)
```

- TF: Term Frequency
- IDF: Inverse Document Frequency
- TF-IDF(mutiplication)

```python
tf_idf_matrix = np.array([
    [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0],
 [0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
 [0.0, 0.0, 0.0, 0.0, 0.0, 0.707107, 0.707107, 0.0, 0.0, 0.745636],
 [0.0, 0.0, 0.0, 0.666353, 0.0, 0.0, 0.0, 0.0, 0.0, 0.745636],
 [0.0, 0.0, 0.0, 0.666353, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
 [1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.830083],
 [0.0, 0.0, 0.0, 0.557639, 0.0, 0.0, 0.0, 0.0, 0.707107, 0.0],
 [0.0, 0.0, 0.0, 0.0, 0.707107, 0.0, 0.0, 0.0, 0.0, 0.0],
 [0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
 [0.0, 0.0, 0.0, 0.666353, 0.0, 0.0, 0.0, 0.0, 0.0, 0.745636],
 [0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
])
```

It should be 3 matrixes with 1 by 4, instead of on matrix with 3 by 4

because we have 3 feedbackes and we use different matrixes to divide the 3 feedbacks.

A new data to try again

```python
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

# Correctly formatted feedbacks with spaces after commas to ensure proper tokenization
feedbacks = [
["Apple", "Banana", "Cherry", "Date", "Elderberry"],
["Fig", "Apple", "Grape", "Honeydew melon", "Kiwi"],
["Lemon", "Mango", "Banana", "Orange", "Papaya"],
["Quince", "Raspberry", "Strawberry", "Apple", "Tomato"],
["Ugli fruit", "Vanilla", "Watermelon", "Xigua", "Banana"],
["Yellowtail", "Zucchini", "Almonds", "Apple", "Bacon"],
["Cod", "Duck", "Eel", "Fig", "Flounder"],
["Grouper", "Halibut", "Ice cream", "Banana", "Kale"],
["Lamb", "Mussels", "Noodles", "Oysters", "Pork"],
["Quail", "Ribs", "Salmon", "Tuna", "Urchin"],
["Veal", "Waffles", "Xacuti", "Yogurt", "Ziti"],
["Artichoke", "Broccoli", "Carrot", "Donut", "Apple"],
["Eggplant", "French fries", "Green beans", "Hamburger", "Banana"],
["Icing", "Jelly", "Ketchup", "Lasagna", "Meatballs"],
["Nachos", "Olive oil", "Pizza", "Quiche", "Raspberry"],
["Roast beef", "Sushi", "Turkey", "Upside-down cake", "Vinegar"],
["Wonton", "Xiaolongbao", "Yam", "Za'atar", "Apple"],
["Avocado", "Brisket", "Cheese", "Dumpling", "Escargot"],
["Falafel", "Gnocchi", "Hot dog", "Indian curry", "Banana"],
["Jalapeno", "Kimchi", "Lobster", "Macaroni", "Nutella"],
]
```

Creat a new dataset by ourselves

we have 20 people with different hoobies(food preference)

Dimensionality Reduction

Use SVD to reduce the dimensions of the data

```
# Correcting the structure of the TF-IDF matrix and attempting SVD again
tf_idf_matrix = np.array([
    [0.000000,0.321899,0.000000,0.000000,0.000000,0.321899,0.000000,0.000000,0.000000,0.0000
[0.000000,0.306841,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.000000,0.000000,0.000000,0.000000,0.298794,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.306841,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.000000,0.000000,0.000000,0.000000,0.298794,0.000000,0.000000,0.000000,0.000000,0
[0.477159,0.298794,0.000000,0.000000,0.477159,0.000000,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.000000,0.000000,0.000000,0.000000,0.298794,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.298794,0.477159,0.000000,0.000000,0.000000,0.000000,0.477159,0.477159,0.000000,0
[0.000000,0.000000,0.000000,0.000000,0.000000,0.298794,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.298794,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.000000,0.000000,0.447214,0.000000,0.000000,0.447214,0.000000,0.000000,0.447214,0
[0.000000,0.000000,0.000000,0.000000,0.000000,0.298794,0.000000,0.000000,0.000000,0.000000,0
[0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0.000000,0
])
# Perform SVD
U, s, Vt = np.linalg.svd(tf_idf_matrix, full_matrices=False)

# The singular values are returned in a 1-D array, for full reconstruction they need to be i
Sigma = np.diag(s)

# Provide the outputs
```

Matrix U: T
vectors are c
transformed

Matrix Σ: Th
The singular
direction (*re*
are typically

Matrix Vt: T
to U, these v
feature space

Neighborhood Formation

Similarity and K-NNG

NOW WE ARE CORRET!
IT HAS 20 products, which means we have 20 feedbacks

```
Product 0 is in neighborhood with: [0, 1, 3]
Product 1 is in neighborhood with: [0, 1, 6]
Product 2 is in neighborhood with: [0, 2, 4]
Product 3 is in neighborhood with: [0, 3, 14]
Product 4 is in neighborhood with: [0, 2, 4]
Product 5 is in neighborhood with: [0, 1, 5]
Product 6 is in neighborhood with: [0, 1, 6]
Product 7 is in neighborhood with: [0, 2, 7]
Product 8 is in neighborhood with: [0, 1, 8]
Product 9 is in neighborhood with: [0, 1, 9]
Product 10 is in neighborhood with: [0, 1, 10]
Product 11 is in neighborhood with: [0, 1, 11]
Product 12 is in neighborhood with: [0, 2, 12]
Product 13 is in neighborhood with: [0, 1, 13]
Product 14 is in neighborhood with: [0, 3, 14]
Product 15 is in neighborhood with: [0, 1, 15]
Product 16 is in neighborhood with: [0, 1, 16]
Product 17 is in neighborhood with: [0, 1, 17]
Product 18 is in neighborhood with: [0, 2, 18]
Product 19 is in neighborhood with: [0, 1, 19]
```

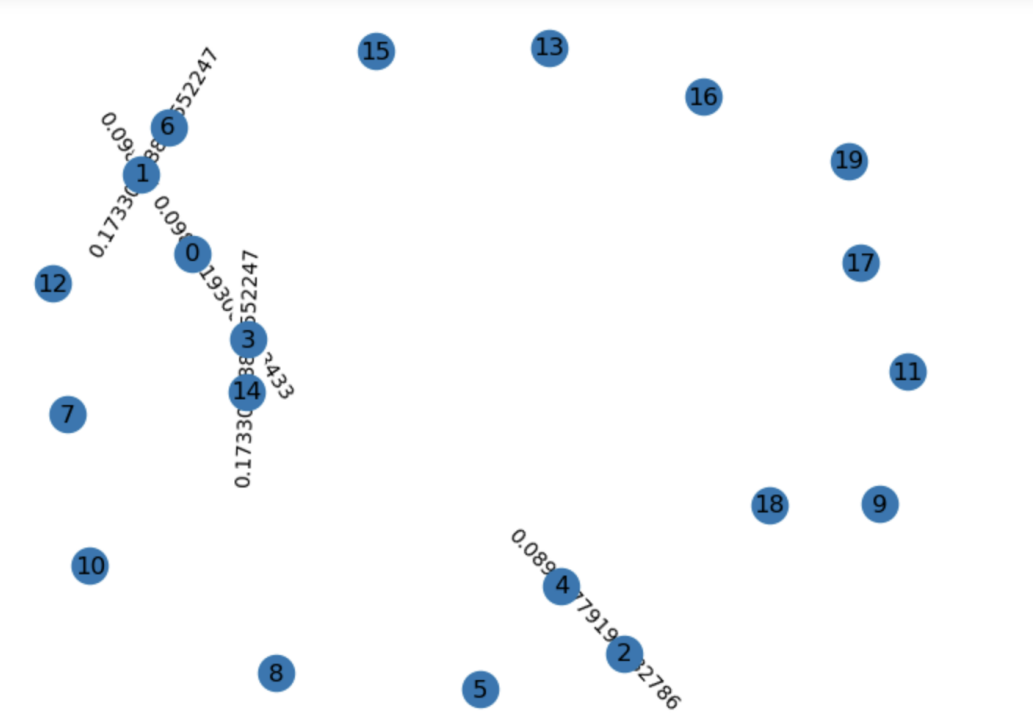
```
Adjacency Matrix:
[[1. 1. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 1. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 0. 1. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
 [1. 0. 1. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 1. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 1. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 0. 1. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 1. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 1. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
 [1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
 [1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
 [1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
 [1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0.]
 [1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
```

```
Similarity Matrix:
[[1.         0.09877193 0.09618152 0.09877193 0.0961
  0.         0.09618152 0.         0.         0.
  0.09618152 0.         0.         0.         0.0961
  0.09618152 0.        ]
 [0.09877193 1.         0.         0.0941515  0.
  0.17330689 0.         0.         0.         0.
  0.         0.         0.         0.         0.0916
  0.         0.        ]
 [0.09618152 0.         1.         0.         0.08927779 0.
  0.         0.08927779 0.         0.         0.         0.
  0.08927779 0.         0.         0.         0.         0.
  0.08927779 0.        ]
 [0.09877193 0.0941515  0.         1.         0.         0.09168227
  0.         0.         0.         0.         0.         0.09168227
  0.         0.         0.17330689 0.         0.09168227 0.
  0.         0.        ]
```

Graph Generation

- Each node represents a product.
- The weight of each edge wij in the graph is equal to the computed similarity between the respective product vectors which are generated from the review text and stored in adjacency matrix A
- If the corresponding entry in the adjacency matrix is non-zero, an edge is created between the two nodes.

```python
# Create a weighted graph
def create_weighted_graph(adjacency_matrix, similarity_matrix):
    # Create an empty weighted graph
    G = nx.Graph()

    # Add nodes to the graph
    num_nodes = adjacency_matrix.shape[0]
    for i in range(num_nodes):
        G.add_node(i)

    # Traverse the adjacency matrix and add edges to the graph
    for i in range(num_nodes):
        for j in range(i+1, num_nodes):  # Only traverse the upper triangle part to reduce r
            if adjacency_matrix[i, j] != 0:
                weight = similarity_matrix[i, j]  # Use the similarity from the similarity m
                G.add_edge(i, j, weight=weight)
    return G
# Visualize or use the graph
def visualize_graph(G):
    # Visualize the graph using NetworkX's built-in drawing function
    nx.draw(G, with_labels=True)
    # Display the graph
    plt.show()

# Generate the weighted graph
weighted_graph = create_weighted_graph(adjacency_matrix, similarity_matrix)

# Visualize the weighted graph
visualize_graph(weighted_graph)
```



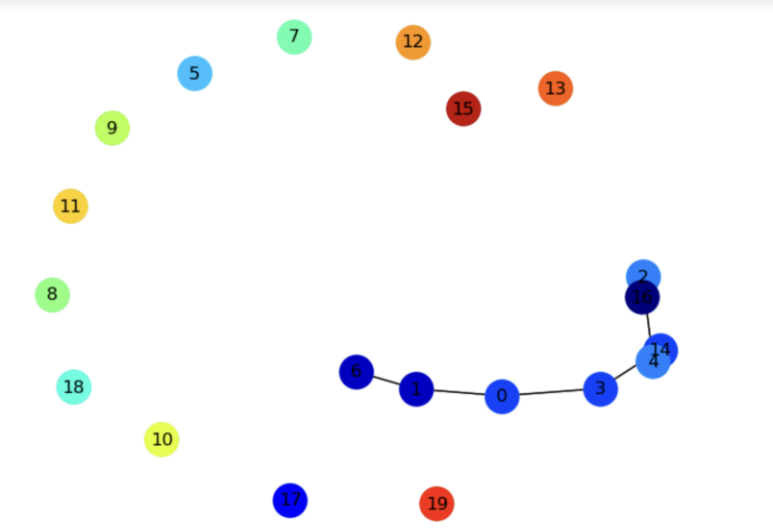
Commuity Detection

- Create different communities of products based on their similarities, as identified by the respective edge weights w_{ij} between nodes.
- **K_Clique_Communities**

① Define k value: Firstly, a parameter k needs to be specified, the number of nodes in the subgraph.
② Search for k-cliques: The algorithm traverses each node in the graph and attempts to expand it into a clique of size k. To achieve this, it examines the neighbors of the node to determine if there exist k-1 other nodes connected to it, forming a complete subgraph.
③ Merge overlapping cliques: Once all size-k cliques are found, the algorithm checks for overlaps between them and merges cliques with higher overlap into larger communities. This step ensures that the communities identified by the algorithm exhibit greater connectivity and coherence.
④ output results: Finally, the algorithm returns the discovered communities as output, where each community is a set of nodes with highly cohesive connections between them.

```python
import community  # Import the community library, which provides an implementation of the Lo
import networkx as nx
import matplotlib.pyplot as plt

# Assuming you have created a weighted graph called weighted_graph

# Perform community detection using the Louvain method
partition = community.best_partition(weighted_graph)

# Group nodes according to their communities
communities = {}
for node, community_id in partition.items():
    if community_id not in communities:
        communities[community_id] = []
    communities[community_id].append(node)

# Output the list of nodes in each detected community
for community_id, nodes in communities.items():
    print(f"Community {community_id}: {nodes}")

# Visualize the weighted graph and label each community with a different color
pos = nx.spring_layout(weighted_graph)  # Define the layout of the graph
nx.draw(weighted_graph, pos, with_labels=True, node_color='lightblue', node_size=500)
for community_id, nodes in communities.items():
    nx.draw_networkx_nodes(weighted_graph, pos, nodelist=nodes, node_color=plt.cm.jet(commun
nx.draw_networkx_edges(weighted_graph, pos, width=1.0, alpha=0.5)
```

```
Community 3: [0, 3, 14]
Community 1: [1, 6]
Community 4: [2, 4]
Community 5: [5]
Community 7: [7]
Community 8: [8]
Community 9: [9]
Community 10: [10]
Community 11: [11]
Community 12: [12]
Community 13: [13]
Community 15: [15]
Community 0: [16]
Community 2: [17]
Community 6: [18]
Community 14: [19]
```



Ranking Products within Graph Communities

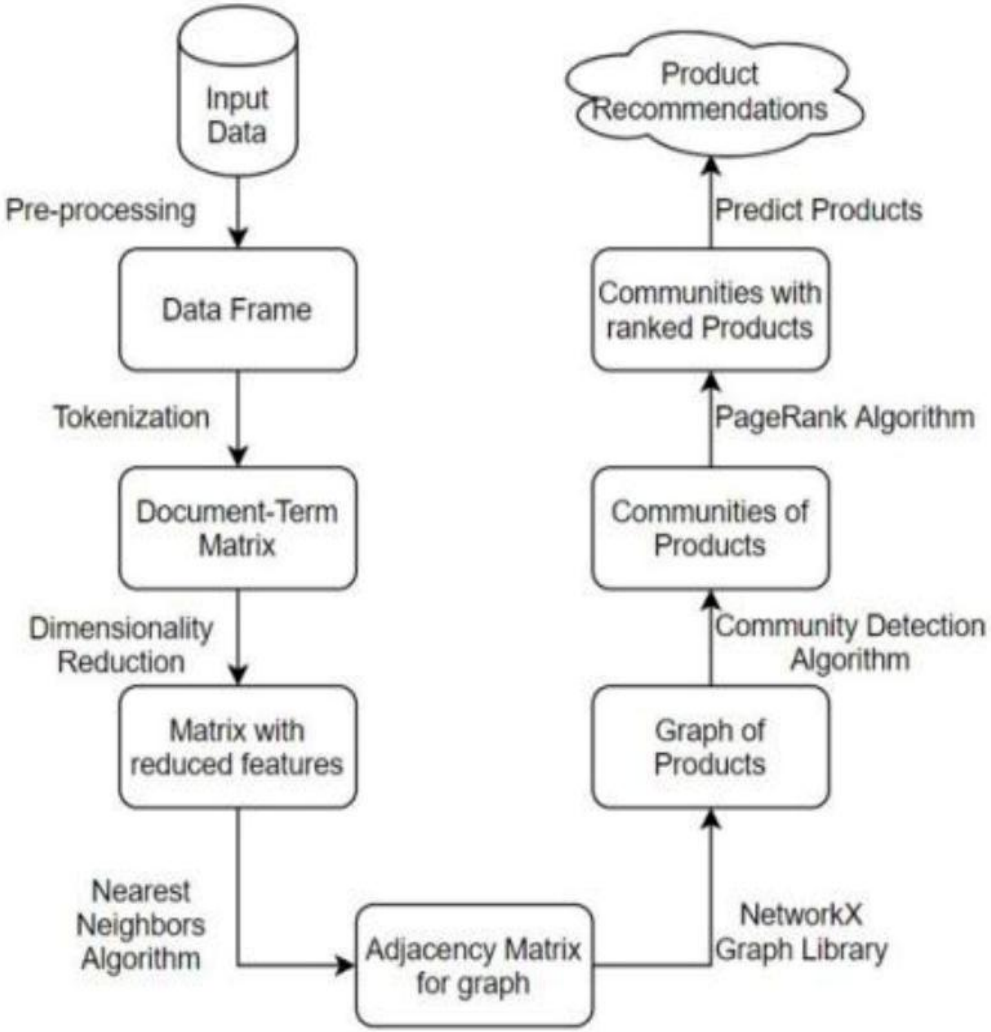
PageRank Algorithm

determine the importance or popularity of products. PageRank is a network ranking algorithm originally used by Google to evaluate the importance of web pages based on the link structure between them.

```python
# Run the PageRank algorithm
pagerank_scores = nx.pagerank(weighted_graph)

# Calculate personalized PageRank scores for each product within its community
personalized_ranks = {}
for community_id, nodes in communities.items():
    for node in nodes:
        # Here, we assume the community ID represents some kind of similarity weight
        personalized_ranks[node] = pagerank_scores[node] * partition[node]

# Output the personalized rank for each product
for node, rank in personalized_ranks.items():
    print(f"Product {node} has a personalized rank of: {rank}")

plt.show()
```

```
Product 0 has a personalized rank of: 0.10155707604417598
Product 1 has a personalized rank of: 0.13739791083442857
Product 6 has a personalized rank of: 0.09115316008201087
Product 2 has a personalized rank of: 0.4469273743016439
Product 4 has a personalized rank of: 0.4469273743016439
Product 3 has a personalized rank of: 0.4121937325032857
Product 14 has a personalized rank of: 0.2734594802460326
Product 5 has a personalized rank of: 0.08379888268158597
Product 7 has a personalized rank of: 0.11731843575422038
Product 8 has a personalized rank of: 0.13407821229053757
Product 9 has a personalized rank of: 0.15083798882685476
Product 10 has a personalized rank of: 0.16759776536317195
Product 11 has a personalized rank of: 0.18435754189948916
Product 12 has a personalized rank of: 0.20111731843580635
Product 13 has a personalized rank of: 0.21787709497212354
Product 15 has a personalized rank of: 0.2513966480447579
Product 16 has a personalized rank of: 0.0
Product 17 has a personalized rank of: 0.03351955307263439
Product 18 has a personalized rank of: 0.10055865921790318
Product 19 has a personalized rank of: 0.23463687150844076
```

Conclusion



Some thoughts about the project:

1. actually we fail to use a real data, because of some complexed problems(we have mentioned before)

2. as we all know , “graph based” is better than other ways because it focus on similarity and potential, but just because its advantage, something will become very difficult, <u>just like the first step: Pre-processing.</u>

3. We still need to study some new things like trying to use <u>NTLK package and spaCy package</u> to grab keywords and try to find some real dataset with food names and positve/negative comments